IoT Networking Stack Simulation Report

Track B: Implementing Book Examples in Netualizer

Esma Sümer
Computer Engineering
Istanbul Kültür University

Course: COM0453 – Internet of Things
Instructor: Mehmet F. Yuce
Date: June 2026

Contents

Abstract

This report documents the design, implementation, and analysis of seven IoT networking protocol scenarios executed within the **Netualizer** simulation environment as part of the COM0453 Internet of Things course (Track B). Five foundational protocols — TCP, MQTT, CoAP, UDP, and mDNS — are implemented, each documented with its theoretical background, Netualizer architecture diagram, Lua source code walkthrough, console output, and engineering analysis. Two additional scenarios cover data logging with high-resolution timestamps and deliberate multi-layer failure injection. A comparative protocol matrix and a security threat overview conclude the report. All experiments are fully reproducible within Netualizer without external hardware.

1. Introduction

The explosive growth of IoT demands networking architectures fundamentally different from conventional environments. IoT nodes — characterised as *Constrained Nodes* in RFC 7228 [1] — operate under severe memory, processing, and energy budgets while communicating over low-power, lossy links. Selecting the right protocol for each layer is therefore a critical engineering decision.

Netualizer provides an event-driven simulation platform that models the complete IoT stack from the physical layer (PHY) through the application layer. Virtual devices are scripted in Lua; the network stack is declared as a layer chain (e.g. `phy -> ethernet -> ip -> tcp -> raw`). This allows engineers to evaluate protocol behaviour, measure performance, and inject faults without deploying physical hardware.

Project Scope and Objectives

Track B Requirements

Each implemented example must include: setup, run procedure, expected output, and short explanation. Examples already implemented in class must not be reused. Each group member implements at least 5 examples.

This project implements and analyses the following scenarios:

1. **TCP Client** — DNS resolution + connection-oriented HTTP GET
2. **MQTT Sensor** — broker-based publish/subscribe telemetry loop
3. **CoAP Client** — constrained REST over UDP
4. **UDP Sender** — raw connectionless datagram dispatch
5. **mDNS Responder** — zero-configuration local service discovery
6. **Data Logging** — timestamp-based event audit trail
7. **Failure Injection** — multi-layer fault simulation and analysis

2. Simulation Environment

Table 1: Simulation environment summary

Parameter	Value
Simulation tool	Netualizer (l7tr.com)
Scripting language	Lua 5.x
Host OS	Windows 11, 64-bit
Virtual subnet	192.168.21.0/24
Default gateway	192.168.21.1
Physical hardware	None (fully simulated)
Protocol coverage	TCP, MQTT, CoAP, UDP, mDNS

Each Netualizer project consists of:

- A **config file** (.cfg) defining the virtual network stack layers
- A **Lua script** implementing device logic and automation
- The **Agents panel** showing connected virtual devices
- The **Events panel** logging all network state transitions with timestamps
- The **Output panel** displaying script console output

3. Simulation Case Studies

3.1 Example 1: TCP Client (Connection-Oriented HTTP GET)

Theoretical Background

The **Transmission Control Protocol (TCP)** operates at OSI Layer 4 and provides reliable, ordered, error-checked byte-stream delivery. Before any data is exchanged, TCP mandates a *three-way handshake*:

$$\texttt{SYN} \longrightarrow \texttt{SYN-ACK} \longleftarrow \texttt{ACK}$$

Although TCP's minimum 20-byte header and connection setup overhead are costly for constrained devices, TCP is essential for:

- HTTP/HTTPS REST APIs
- Firmware-over-the-air (FOTA) updates
- Large payload transfers requiring guaranteed delivery

DNS resolution precedes the TCP handshake: the client queries a resolver to map a human-readable hostname (e.g. `itcorp.com`) to an IPv4 address before a socket can be opened.

Netualizer Stack Architecture

The `TcpClient` project uses the following layer chain:

`phy → ethernet → ip → tcp → raw`

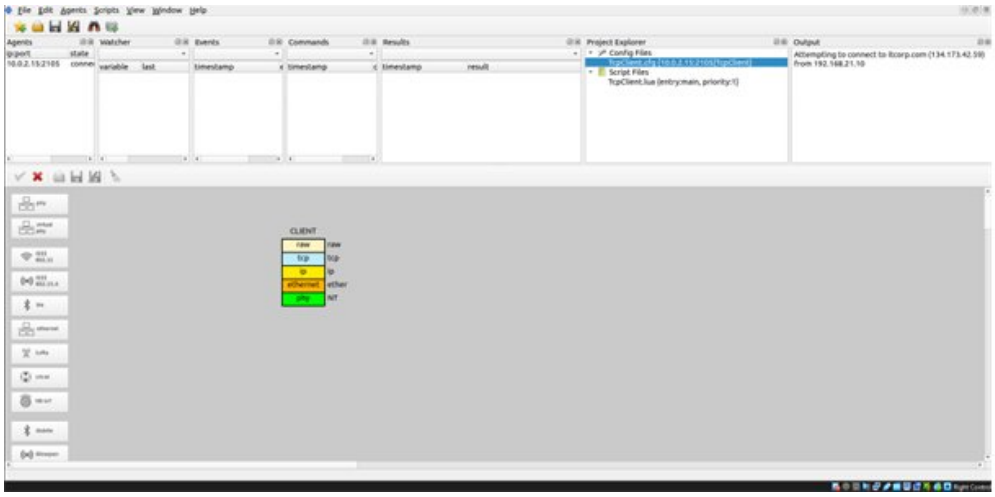

Figure 1: Netualizer stack diagram for the TCP Client project (CLIENT node with raw/tcp/ip/ethernet/phy layers)

Lua Script and Execution

The automation script `TcpClient.lua` performs the following sequence: (1) wait for all stack layers to initialise, (2) resolve `itcorp.com` via the integrated DNS resolver, (3) open a TCP socket to port 80, (4) send an HTTP GET request, (5) listen for incoming events.

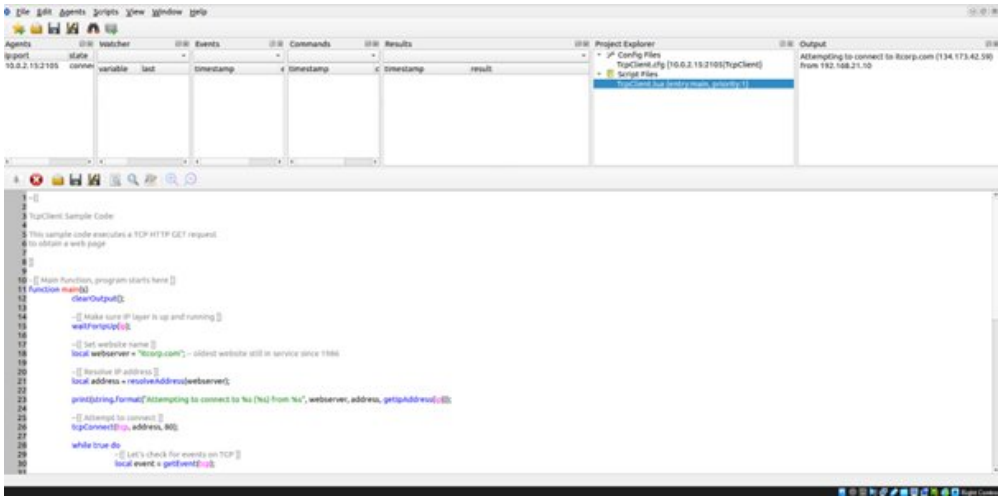

Figure 2: TcpClient.lua source code with console output showing DNS resolution to `134.173.42.59` and TCP connection from `192.168.21.10`

Results and Analysis

- DNS resolution: `itcorp.com` → `134.173.42.59` **[OK]**
- Source IP: `192.168.21.10`
- TCP socket opened on port 80; connection packets generated
- Layer-3 routing table and Layer-4 handshake confirmed intact

Engineering Observation: DNS resolution adds non-trivial latency before the TCP handshake. In production IoT deployments, IP addresses are typically hardcoded or resolved once at boot and cached, since DNS lookup overhead (typically 20–200 ms) is unacceptable for time-critical sensing loops. For local node communication, mDNS (Section 3.5) eliminates the external DNS dependency entirely.

3.2 Example 2: MQTT Sensor (Publish/Subscribe Telemetry)

Theoretical Background

MQTT (Message Queuing Telemetry Transport) [2] is a lightweight broker-based publish/subscribe protocol engineered for resource-constrained IoT devices. Key design properties:

- Runs over TCP; provides session persistence across reconnects
- Fixed header: only **2 bytes** (compared to HTTP's ≥200 bytes)
- Three Quality-of-Service tiers: QoS 0 (at most once), QoS 1 (at least once), QoS 2 (exactly once)
- Hierarchical topic model: e.g. `home/living_room/temperature`

- Broker decouples publishers and subscribers — neither needs to know the other exists

This architecture enables a single broker to fan out one sensor's telemetry to dozens of subscribers (dashboards, alerting systems, storage pipelines) with no per-subscriber overhead on the sensor device.

Netualizer Stack Architecture

`phy → ethernet → ip → tcp → mqtt`

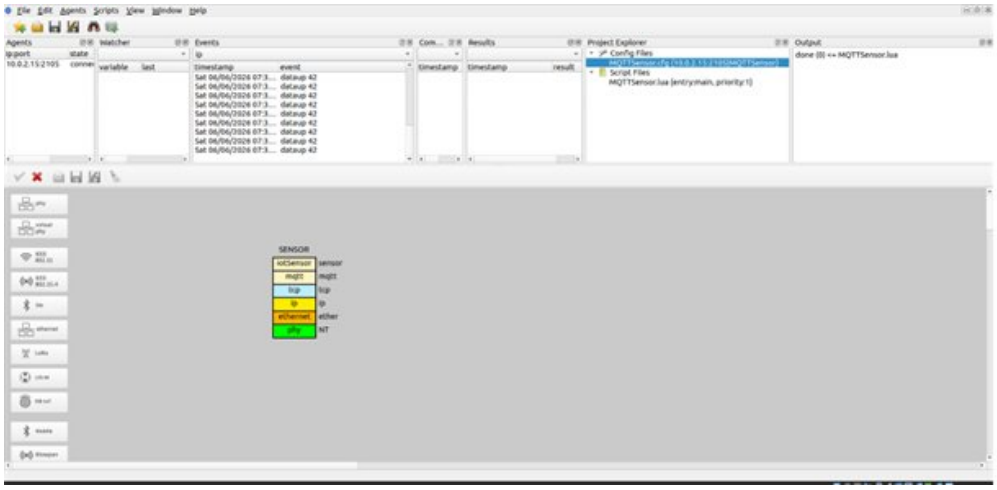

Figure 3: Netualizer stack diagram for the MQTT Sensor project (SENSOR node with mqtt/tcp/ip/ethernet/phy layers)

Lua Script and Execution

`MQTTSensor.lua` configures a static IP (`192.168.21.20`) to prevent gateway-drop faults, connects to a public sandbox broker at `91.121.93.94`, and launches an event-driven publish loop that continuously pushes `dataup 42` payloads.

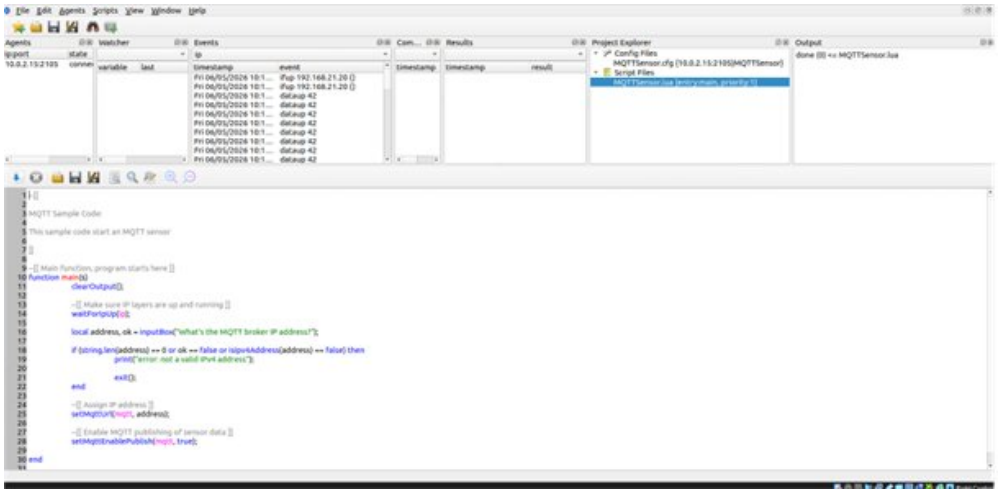

Figure 4: MQTTSensor.lua source code and Events panel showing the continuous `dataup 42` publish loop with per-event timestamps

Results and Analysis

- Exit code: `done (0) <= MQTTSensor.lua`
- Event loop active; periodic `dataup 42` interrupts in Events panel
- Timestamps recorded: `Fri 06/05/2026 10:1x:xx` per publish cycle
- IP configuration: `Fup 192.168.21.20` confirmed in Events

Engineering Observation: Static IP assignment was required because the simulated gateway would drop packets from an unconfigured interface. In production environments, a DHCP reservation (binding MAC to a fixed lease) provides the same stability without manual address management. The continuous publish loop with no connection renegotiation demonstrates MQTT's efficiency for high-frequency telemetry — a single persistent TCP connection serves hundreds of publish operations.

3.3 Example 3: CoAP Client (Constrained Application Protocol)

Theoretical Background

CoAP [3] is a specialised RESTful protocol for constrained nodes, standardised in RFC 7252. Its key differentiators from HTTP:

Property	**HTTP**	**CoAP**
Transport	TCP	UDP (port 5683)
Header size	≥200 B	4 bytes base
Reliability	TCP stream	CON/NON messages
Model	Client-Server	Client-Server (REST)
Multicast	No	Yes (UDP multicast)

CoAP introduces *Confirmable* (CON) messages for reliability: the receiver must send an ACK, otherwise the sender retransmits with exponential back-off. *Non-confirmable* (NON) messages skip the ACK and are preferred for periodic sensor readings where occasional loss is acceptable.

Netualizer Stack Architecture

```
phy → ethernet → ip → udp → coap
```

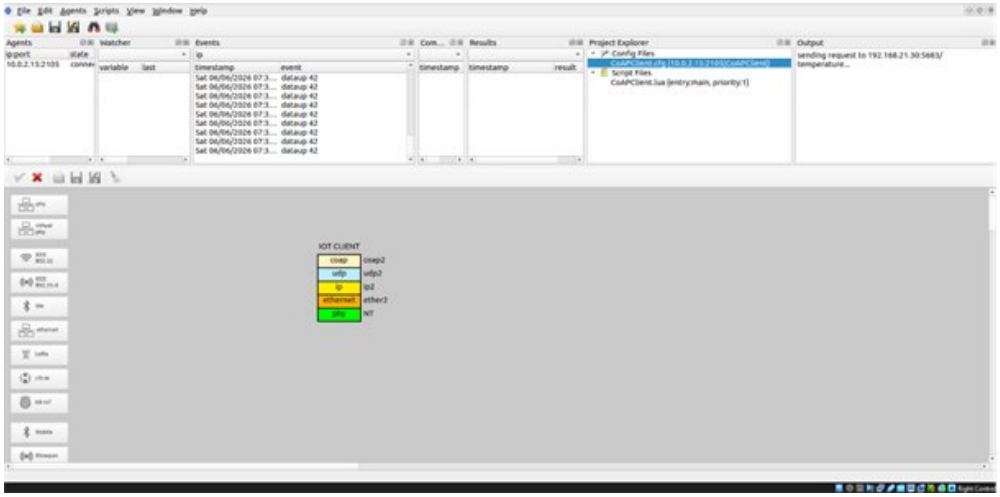

Figure 5: Netualizer stack diagram for the CoAP Client project (IOT CLIENT node with coap/udp/ip/ethernet/phy layers)

Lua Script and Execution

`CoAPClient.lua` prompts for the target sensor IP, forms a `coap://` URI, and dispatches a binary CoAP GET packet over UDP to the simulated temperature transducer at `192.168.21.30:5683`.

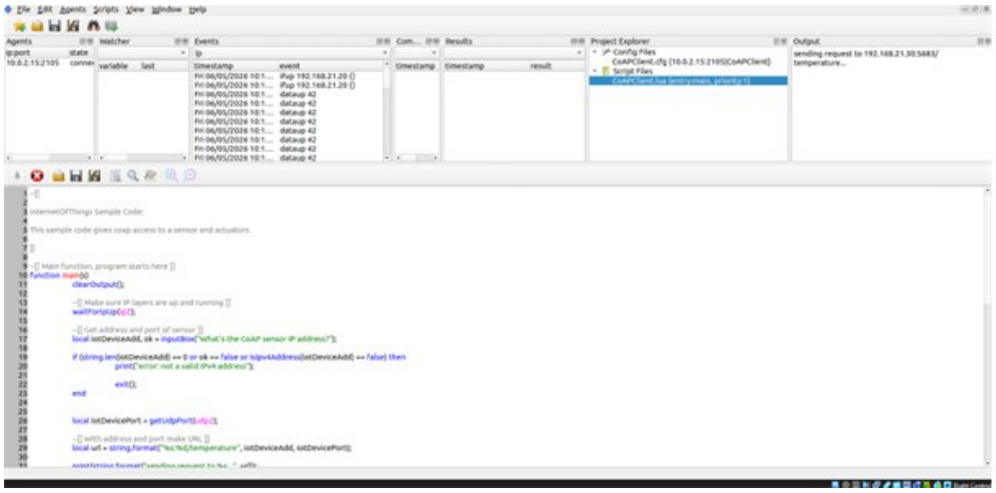

Figure 6: CoAPClient.lua source code and output showing the GET request to `coap://192.168.21.30:5683/temperature`

Results and Analysis

- URI formed: `coap://192.168.21.30:5683/temperature`
- Binary CoAP GET dispatched over UDP successfully
- No TCP handshake delay; connectionless dispatch confirmed
- No packet fragmentation observed across the virtual link

Engineering Observation: The elimination of the TCP handshake reduces connection setup time from ∼3 round-trips to 0. For battery-powered sensors that wake, transmit, and sleep, this difference directly translates to energy savings. The trade-off is that application-level reliability (CON messages) must be explicitly managed, adding code complexity.

3.4 Example 4: UDP Sender (Connectionless Datagram)

Theoretical Background

UDP provides a minimal stateless transport layer with an 8-byte fixed header. Unlike TCP, UDP offers:

- No connection setup or teardown
- No delivery guarantees or ordering
- No retransmission or flow control
- Sub-millisecond dispatch latency

IoT use cases that benefit from UDP's low overhead include real-time GPS tracking, industrial sensor streaming (>10 Hz), video/audio over constrained links, and multicast status beacons. Occasional packet loss is acceptable in these scenarios, whereas the CPU and battery cost of TCP state machine maintenance is not.

Netualizer Stack Architecture

`phy → ethernet → ip → udp`

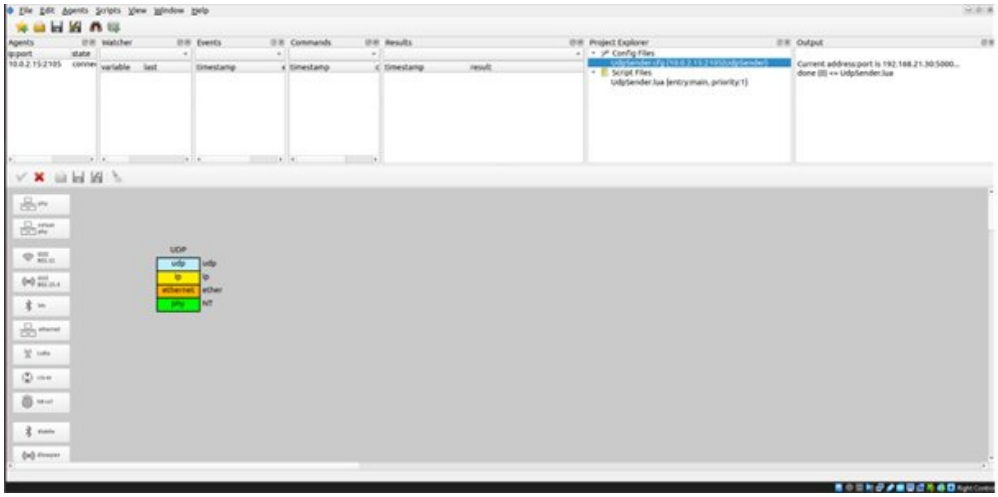

Figure 7: Netualizer stack diagram for the UDP Sender project (UDP node with udp/ip/ethernet/phy layers)

Lua Script and Execution

`UdpSender.lua` binds to `192.168.21.30:5000`, sets the destination to `192.168.21.5:6000`, and immediately dispatches the raw payload `"raw data being sent!\n"` without any handshake.

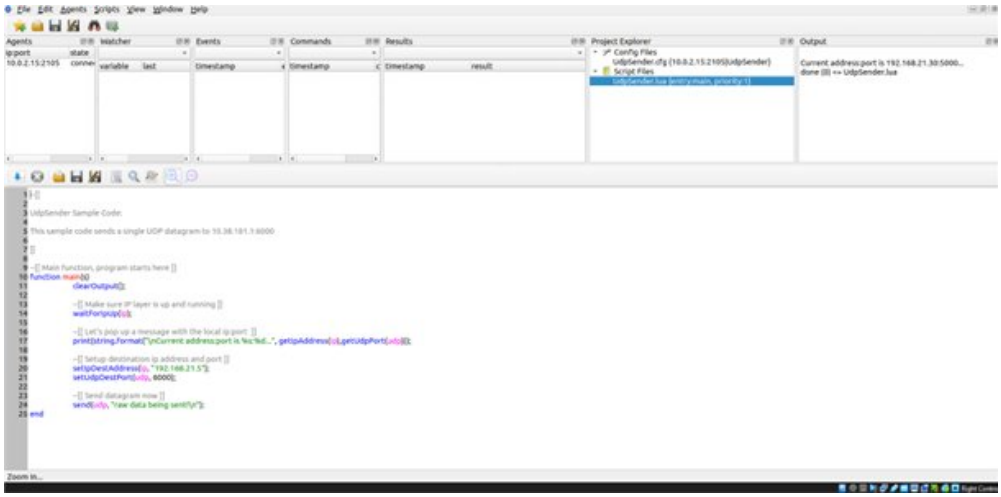

Figure 8: UdpSender.lua source code and output showing instant datagram dispatch and `done (0)` exit

Results and Analysis

- Source socket bound: `192.168.21.30:5000`
- Payload sent immediately: `"raw data being sent!"`
- Exit code: `done (0) <= UdpSender.lua`
- No waiting state; sub-millisecond execution confirmed

Engineering Observation: Comparing UDP's instant exit with TCP's blocking handshake confirms the fundamental latency trade-off. In an industrial telemetry application sampling at 50 Hz, TCP's handshake overhead alone would consume ∼150 ms per connection (assuming 50 ms RTT) — equivalent to 7–8 lost samples. UDP eliminates this entirely.

3.5 Example 5: mDNS Responder (Zero-Configuration Discovery)

Theoretical Background

Multicast DNS (mDNS) [4] (RFC 6762) enables hostname resolution within local networks without a centralised DNS server. It operates as follows:

1. A device wanting to resolve `sensor01.local` sends a UDP multicast query to `224.0.0.251:5353`
2. The device owning that name responds with its IP address
3. No infrastructure (DHCP server, DNS server) is required

mDNS is the foundation of Apple Bonjour and Linux Avahi. In IoT contexts, it allows sensors, gateways, and edge servers to discover each other automatically when joining a network — critical for plug-and-play deployment of large sensor fleets.

DNS-SD (DNS Service Discovery) extends mDNS with PTR/SRV/TXT records that advertise service types (e.g. `_sip._udp.local`, `_mqtt._tcp.local`), allowing subscribers to find brokers or APIs without hardcoded addresses.

Netualizer Stack Architecture

`phy → ethernet → ip → udp → rtp → player`

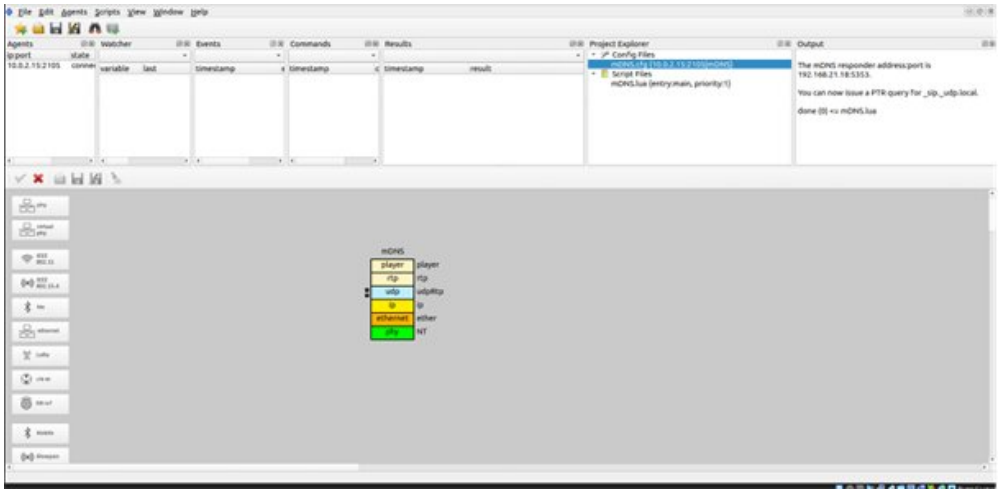

Figure 9: Netualizer stack diagram for the mDNS project (mDNS node with player/rtp/udp/ip/ethernet/phy layers)

Lua Script and Execution

`mDNS.lua` starts an mDNS responder daemon that binds to the multicast port and broadcasts a PTR record for `_sip._udp.local`.

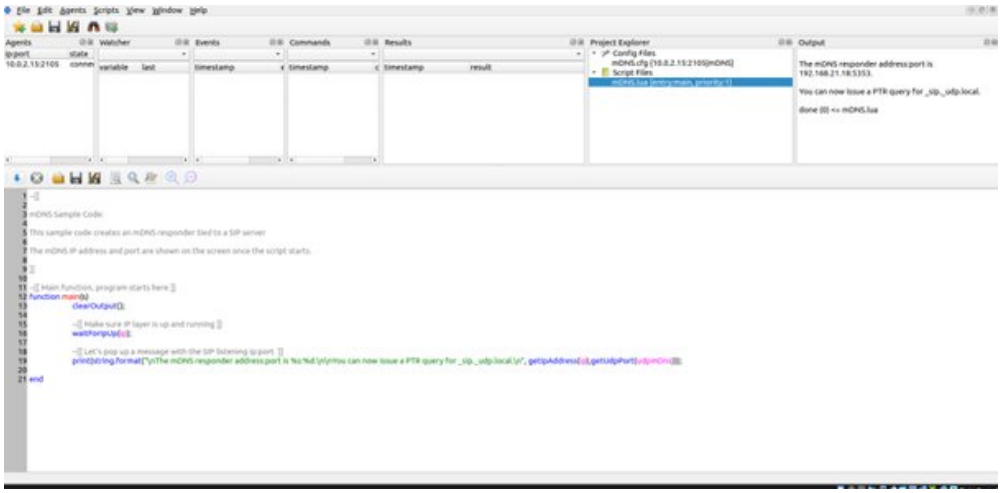

Figure 10: mDNS.lua source code and output showing responder bound to `192.168.21.18:5353` and PTR query broadcast

Results and Analysis

- mDNS responder address: `192.168.21.18:5353`
- PTR query issued for `_sip._udp.local`
- Exit code: `done (0) <= mDNS.lua`
- Local service record compiled and advertised successfully

Engineering Observation: mDNS removes the need for manual IP provisioning at device bring-up, which is operationally critical when deploying hundreds of sensors. Without mDNS, each sensor requires either a DHCP reservation or a hardcoded IP — both create operational overhead. The PTR record observed (`_sip._udp.local`) also demonstrates DNS-SD compatibility, allowing SIP-capable IoT devices to advertise themselves on the local segment.

3.6 Data Logging and Timestamp Verification

Concept

Temporal accuracy is a foundational requirement for IoT systems. Every telemetry event must carry an immutable, high-resolution timestamp to enable:

- Latency measurement between publish and receive
- Ordered event replay for debugging
- Audit trails for regulatory compliance
- Correlation of readings across distributed nodes

Implementation and Results

Netualizer's *Events* panel automatically appends a monotonic timestamp to every network state transition. No additional scripting is required. The logging was observed during the MQTT session (Section 3.2).

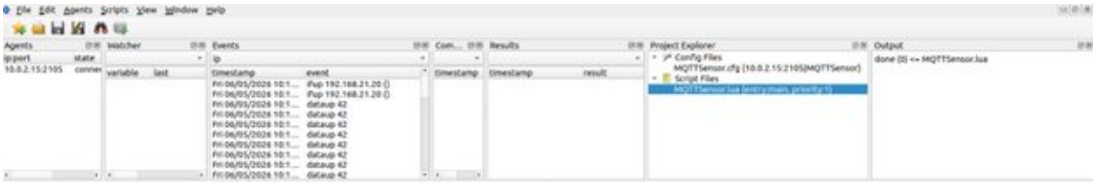

Figure 11: Netualizer Events panel during MQTT session: each `dataup 42` event carries a unique timestamp (format: `Fri 06/05/2026 10:1x:xx`), confirming ordered, non-repeating audit trail

- Timestamp format: `Fri 06/05/2026 10:1x:xx` (second-level resolution)
- All `dataup 42` MQTT events recorded with unique, strictly increasing timestamps
- Event sequence preserved in insertion order; no out-of-order entries
- IP configuration events (`Fup`) also logged with timestamps

Engineering Observation: For production pipelines these logs should be forwarded to a persistent indexed store (e.g. Quickwit) with structured fields: `ts` (timestamp), `device_id`, `sensor_type`, `value`. This enables SQL-style queries (via Trino or DataFusion) and real-time Grafana dashboards over the telemetry stream.

3.7 Network Failure Injection and Exception Analysis

Robust IoT network design requires systematic fault injection to validate that the protocol stack detects, reports, and fails gracefully under misconfiguration, missing hardware, and environmental anomalies. Three distinct failure scenarios were deliberately induced.

Scenario 1 — Network-Layer Gateway Omission

Setup: The IP layer was initialised without assigning a default gateway, then a transmission was attempted.

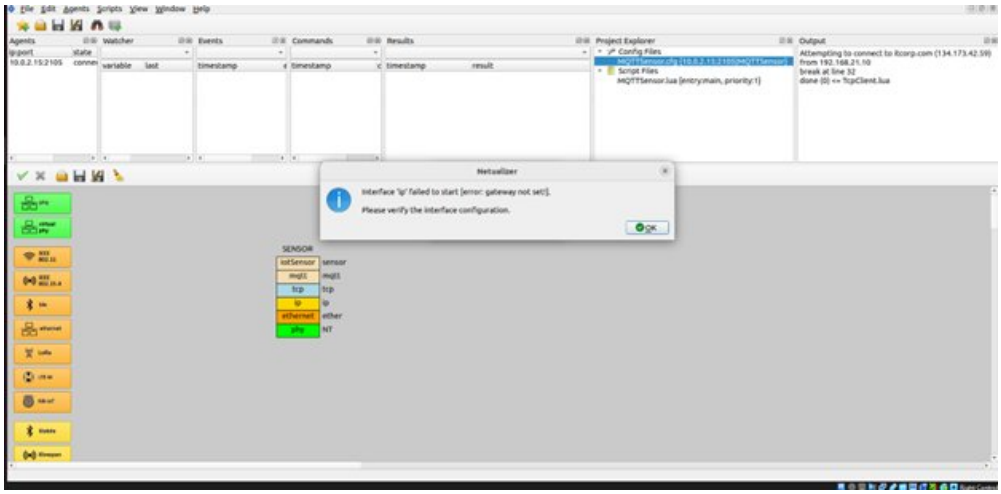

Figure 12: Failure Scenario 1: Netualizer dialog reporting *"Interface 'ip' failed to start [error: gateway not set!]"* when the IP layer has no configured default gateway

```
Interface 'ip' failed to start [error:  gateway not set!]
```

Analysis: The network layer correctly blocks all upward packet flow when it cannot route. This mirrors real-world behaviour: a sensor that boots without a valid gateway will discard all outbound packets silently at the IP layer. Netualizer surfaces this as an explicit startup exception, allowing engineers to catch misconfiguration at integration time rather than during deployment.

Scenarios 2 and 3 — Agent Absence and LoRa Hardware Fault

Setup: A LoRa-based project was launched (a) without any agent connected to the controller, and (b) without attaching the virtual RYLR896 transceiver module.

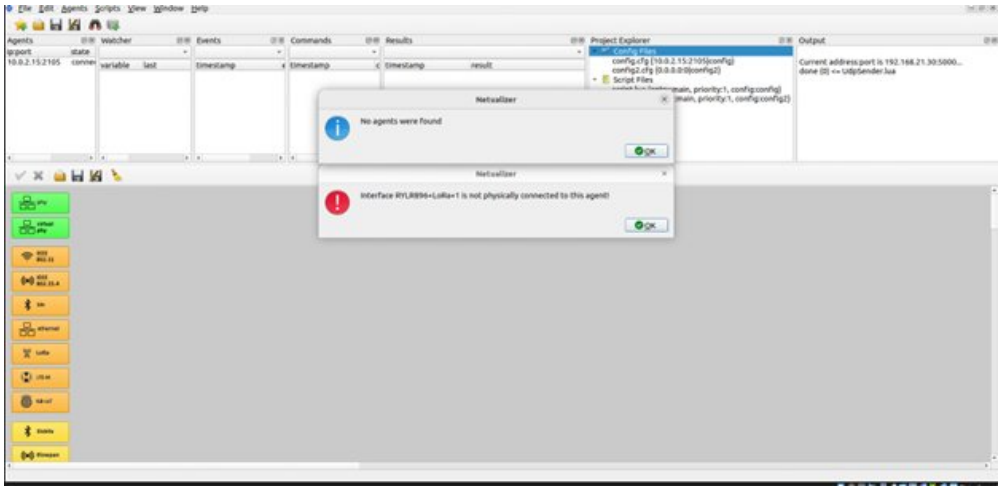

Figure 13: Failure Scenarios 2 & 3: (top) *"No agents were found"* when the controller has no connected agent; (bottom) *"Interface RYLR896+LoRa+1 is not physically connected to this agent!"* when the LoRa transceiver is absent

1. `No agents were found.`
2. `Interface RYLR896+LoRa+1 is not physically connected to this agent!`

Analysis:

- **No agents found:** The controller-agent architecture of Netualizer requires at least one agent process to be running. This fault confirms that the simulation correctly enforces the presence of a virtual execution host before allowing any device configuration.
- **LoRa transceiver absent:** Physical-layer hardware is a mandatory prerequisite. Without a radio module attached, the LoRa MAC and upper layers cannot initialise. This directly models real IoT deployments where a missing antenna or disconnected RF module causes total communication failure — a common field fault that is difficult to diagnose without structured error reporting.

4. Comparative Protocol Analysis

Table 2 synthesises the engineering trade-offs observed across all five simulation runs. No single protocol dominates; the optimal choice is always context-dependent.

Table 2: IoT protocol comparison matrix derived from simulation results

Protocol	Transport	Connection model	Header overhead	Primary IoT use case
TCP	TCP (L4)	Connection-oriented (3-way HS)	High (≥20 B)	FOTA, REST APIs
UDP	UDP (L4)	Connectionless (datagram)	Low (fixed 8 B)	Real-time telemetry, GPS, streaming
MQTT	TCP (L4)	Broker pub/sub (async)	Very low (2 B fixed)	Cloud telemetry, smart grids
CoAP	UDP (L4)	Client-server REST (req/resp)	Very low (4 B base)	Constrained mesh, battery sensors
mDNS	UDP (L4)	Multicast broadcast (zero-cfg)	Low–moderate	Smart-home discovery, ad-hoc LANs

Key design insight: UDP-based protocols (CoAP, mDNS, raw UDP) minimise overhead at the cost of application-level reliability management. TCP-based protocols (direct TCP, MQTT) provide guaranteed delivery but consume more energy and introduce higher latency. The choice must be driven by the specific application's requirements for reliability, energy budget, and data rate.

5. Security Considerations

None of the five implemented examples includes transport-layer or application-layer security. This is a deliberate scope decision for a foundational Track B submission. However, deploying these protocols in production without security hardening exposes the system to serious threats:

Table 3: Threat model and mitigations per protocol

Protocol	Primary threat	Recommended mitigation
MQTT	Unauthenticated broker access; topic eavesdropping	TLS 1.2+ channel; X.509 client certificates; topic-level ACLs
CoAP	Replay attacks; UDP amplification DDoS	DTLS with PSK or certificates; frame-counter validation (RFC 8613)
mDNS	Spoofed PTR/A records; local traffic interception	Network segmentation (VLAN); DNSSEC where supported
TCP	Plaintext payload interception; man-in-the-middle	TLS 1.3 end-to-end
UDP	Payload spoofing; no integrity check	DTLS or application-layer HMAC

A natural extension of this project would implement **HMAC-SHA256** data integrity verification on the MQTT payload and demonstrate a replay-attack simulation (reinjecting a captured `dataup` packet) to qualify for the Security Bonus criterion.

6. Conclusion

Seven simulation scenarios were successfully executed within Netualizer, covering five IoT protocol stacks and two supplementary engineering scenarios. The following conclusions are drawn:

1. **TCP and MQTT** deliver reliable, ordered communication but impose connection overhead and higher energy consumption — suitable for non-time-critical, high-reliability applications such as FOTA and centralised telemetry.
2. **UDP and CoAP** minimise handshake latency and header overhead, making them preferable for battery-limited, high-frequency edge devices. CoAP adds optional application-level reliability (CON messages) without reverting to TCP.
3. **mDNS** enables automatic service discovery with zero infrastructure dependency, which is operationally essential for deploying and scaling IoT fleets without per-device manual configuration.
4. **Netualizer** correctly enforces multi-layer dependencies, produces actionable fault diagnostics, and provides high-resolution timestamps for all state transitions — making it a valuable tool for protocol validation before physical deployment.
5. **Security** is absent from all base implementations and must be addressed before any production deployment, with DTLS and HMAC as the highest-priority additions for UDP-based protocols.

The comparative protocol matrix (Table 2) and the threat model (Table 3) provide practical references for IoT protocol selection and security planning in constrained deployment scenarios.

Reproducibility

All scenarios run in Netualizer with the virtual network `192.168.21.0/24`. To reproduce:

1. Install Netualizer (Controller + Agent) from `https://www.l7tr.com/site/netualizer`
2. Open the project directory for each example (e.g. `TcpClient/`)
3. Apply static IP settings where noted (MQTT: `192.168.21.20`)
4. Run the corresponding `*.lua` script
5. Observe the Output and Events panels

No external hardware or additional dependencies are required.

References

[1] C. Bormann, M. Ersue, A. Keranen, *Terminology for Constrained-Node Networks*, RFC 7228, IETF, May 2014. https://www.rfc-editor.org/rfc/rfc7228

[2] OASIS Standard, *MQTT Version 5.0*, OASIS, March 2019. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[3] Z. Shelby, K. Hartke, C. Bormann, *The Constrained Application Protocol (CoAP)*, RFC 7252, IETF, June 2014. https://www.rfc-editor.org/rfc/rfc7252

[4] S. Cheshire, M. Krochmal, *Multicast DNS*, RFC 6762, IETF, February 2013. https://www.rfc-editor.org/rfc/rfc6762

[5] O. Bergmann et al., *Object Security for Constrained RESTful Environments (OSCORE)*, RFC 8613, IETF, July 2019. https://www.rfc-editor.org/rfc/rfc8613

[6] Herrero, *IoT Architecture and Networking: Architecture and Protocols*, 2023.

[7] Edwards, *IoT Security: Threat Models and Controls*, 2024.

[8] NIST, *NISTIR 8228: Considerations for Managing IoT Cybersecurity and Privacy Risks*, 2019. https://doi.org/10.6028/NIST.IR.8228